Ex.No.6	AI-Generated Code Quality and Correctness Evaluation Framework	URK23CS8018
15-10-2026		

Aim:

To build a Code Quality Evaluation Framework that automatically checks AI-generated Python code using static analysis (Pylint), type checking (Mypy), dynamic testing (Pytest), syntax checking (Python AST), execution testing, quality scoring, and automatic improvement suggestions.

Procedure:

1. Install the required libraries such as pylint, mypy, and pytest using the Python package installer.
2. Import the required Python modules, including AST, OS, RE, SYS, tempfile, subprocess, and textwrap.
3. Define the sample AI-generated Python code containing an addition function.
4. Create test cases with expected outputs to verify the correctness of the generated code.
5. Check the syntax of the Python code using the ast.parse() function.
6. Save the generated code into a temporary Python file for analysis.
7. Perform static code analysis using Pylint to identify coding-style issues and potential errors.
8. Perform type checking using Mypy to identify type-related errors.
9. Execute the generated code and perform dynamic testing by comparing actual outputs with expected outputs.
10. Calculate the overall code quality score out of 100 based on syntax validity, Pylint score, Mypy results, dynamic testing, and documentation.
11. Generate automatic improvement suggestions based on the evaluation results.
12. Display the complete evaluation report, including syntax checking, static analysis, type checking, test results, overall score, and improvement suggestions.

Question:

Build an AI-Generated Code Quality and Correctness Evaluation Framework using Python to automatically evaluate AI-generated code through static analysis using Pylint, type checking using Mypy, dynamic testing using Pytest, syntax checking using Python AST, execution testing, quality scoring, and automatic improvement suggestions.

Program:

```
import ast
import re
```

```
import sys
import tempfile
import subprocess
import os
import textwrap

# AI-generated example code
generated_code = """
def add_numbers(a: int, b: int) -> int:
    \"\"\"Return the sum of two numbers.\"\"\"
    return a + b
"""

# Test cases with expected results
test_cases = [
    ((2, 3), 5),
    ((10, 20), 30),
    ((-2, 2), 0),
    ((0, 0), 0)
]



# Syntax checking using Python AST
def check_syntax(code):
    try:
        ast.parse(code)
        return True, "Syntax is valid."
    except SyntaxError as error:
```

```
     f"Syntax error on line {error.lineno}: {error.msg}"
)



    # Save code into a temporary Python file
    def save_code(code, directory):
    file_path = os.path.join(
     directory, "generated_code.py"
    )

    with open(file_path, "w", encoding="utf-8") as file:
    file.write(textwrap.dedent(code).strip() + "\n")

    return file_path



    # Execute external tools
    def run_tool(command):
    try:
result = subprocess.run(
    command,
    capture_output=True,
    text=True,
    timeout=30
)
return result.stdout + result.stderr, result.returncode
```

```
            except FileNotFoundError:
return f"Tool not found: {command[0]}", 127

    except subprocess.TimeoutExpired:
     return f"Tool timed out: {command[0]}", 124


    # Static analysis using Pylint
    def run_pylint(file_path):
    output, exit_code = run_tool([
     sys.executable, "-m", "pylint",
     "--score=y", "--reports=n", file_path
      ])

    match = re.search(r"rated at ([0-9.]+)/10", output)
    score = float(match.group(1)) if match else None

    return {
"output": output.strip() or "No Pylint output.",
"score_out_of_10": score,
"exit_code": exit_code
    }


    # Type checking using Mypy
    def run_mypy(file_path):
    output, exit_code = run_tool([
     sys.executable, "-m", "mypy",
      "--show-error-codes", file_path
    ])

    return {
     "output": output.strip() or "No Mypy output.",
    "passed": exit_code == 0,
              "exit_code": exit_code
    }
```

```
# Dynamic execution testing
# Use only trusted, self-authored code.
def run_dynamic_tests(code, cases):
    namespace = {}

    try:
        compiled_code = compile(
            code, "<generated_code>", "exec"
        )
        exec(compiled_code, namespace)

        function = namespace.get("add_numbers")

        if not callable(function):
            return {
                "passed": False,
                "passed_count": 0,
                "total": len(cases),
                "details": ["Function add_numbers not found."]
            }

        details = []
        passed_count = 0

        for arguments, expected in cases:
            try:
                actual = function(*arguments)

                if actual == expected:
                    passed_count += 1
                    details.append(
                        f"PASS: add_numbers{arguments} == {expected}"
                    )
                else:
                    details.append(
                        f"FAIL: Expected {expected}, got {actual}"
                    )

            except Exception as error:
                details.append(
                    f"ERROR: {type(error)._name_}: {error}"
                )
```

```
    return {
    "passed": passed_count == len(cases),
    "passed_count": passed_count,
    "total": len(cases),
    "details": details
}
    # Generate improvement suggestions
    def generate_suggestions(
      syntax_valid, pylint_result, mypy_result, tests
    ):
      suggestions = []

      if not syntax_valid:
     return ["Fix syntax errors in the code."]

       score = pylint_result.get("score_out_of_10")

              if score is None or score < 8:
       suggestions.append(
    "Review Pylint warnings and improve code style."
       )

      If not mypy_result.get("passed", False):
               suggestions.append(
    "Fix type checking issues reported by Mypy."
     )

      if tests["passed_count"] < tests["total"]:
                suggestions.append(
    "Fix functional bugs causing tests to fail."
     )

    if not suggestions:
    suggestions.append(
    "Code looks good. Add more test cases if needed."
)

     return suggestions
```

```
# Evaluate code and calculate score
def evaluate_code(code, cases):
    syntax_valid, syntax_message = check_syntax(code)

    result = {
        "syntax_valid": syntax_valid,
        "syntax_message": syntax_message,
        "pylint": {
            "output": "Skipped due to syntax error.",
            "score_out_of_10": None,
            "exit_code": 1
        },
        "mypy": {
            "output": "Skipped due to syntax error.",
            "passed": False,
            "exit_code": 1
        },
        "dynamic_tests": {
            "passed": False,
            "passed_count": 0,
            "total": len(cases),
            "details": ["Skipped due to syntax error."]
        }
    }
```

```
        if syntax_valid:
    with tempfile.TemporaryDirectory() as directory:
        file_path = save_code(code, directory)

        result["pylint"] = run_pylint(file_path)
        result["mypy"] = run_mypy(file_path)
        result["dynamic_tests"] = run_dynamic_tests(
            code, cases
        )

syntax_points = 15 if syntax_valid else 0

pylint_score = result["pylint"]["score_out_of_10"]
pylint_points = (
    20 * pylint_score / 10
    if pylint_score is not None else 0
)

mypy_points = (
    15 if result["mypy"]["passed"] else 0
)

tests = result["dynamic_tests"]

test_points = (
    40 * tests["passed_count"] / tests["total"]
```

```
        if tests["total"] else 0
)

try:
    tree = ast.parse(code)
    has_docstring = any(
        isinstance(
            node,
            (
                ast.FunctionDef,
                ast.AsyncFunctionDef,
                ast.ClassDef
            )
        ) and ast.get_docstring(node)
        for node in ast.walk(tree)
    )
except SyntaxError:
    has_docstring = False

documentation_points = 10 if has_docstring else 0

result["score"] = round(
    syntax_points
    + pylint_points
    + mypy_points
    + test_points
```

```
        + documentation_points,
        1
    )

    result["suggestions"] = generate_suggestions(
        syntax_valid,
        result["pylint"],
        result["mypy"],
        tests
    )

    return result



# Display evaluation report
def print_report(code, result):
    print("=" * 60)
    print("AI-GENERATED CODE QUALITY EVALUATION REPORT")
    print("=" * 60)

    print("\n1. SOURCE CODE")
    print(textwrap.dedent(code).strip())

    print("\n2. SYNTAX CHECK")
    print(result["syntax_message"])
```

```
print("\n3. PYLINT STATIC ANALYSIS")
  print(
     "Score:",
     result["pylint"]["score_out_of_10"],
     "/ 10"
  )
  print(result["pylint"]["output"][:2500])

  print("\n4. MYPY TYPE CHECKING")
  print("Passed:", result["mypy"]["passed"])
  print(result["mypy"]["output"][:2500])

  print("\n5. DYNAMIC TESTING")

  for detail in result["dynamic_tests"]["details"]:
     print(detail)

  print(
     "Tests passed:",
     f"{result['dynamic_tests']['passed_count']}/"
     f"{result['dynamic_tests']['total']}"
  )

  print("\n6. OVERALL DEMONSTRATION SCORE")
  print(f"{result['score']}/100")
```

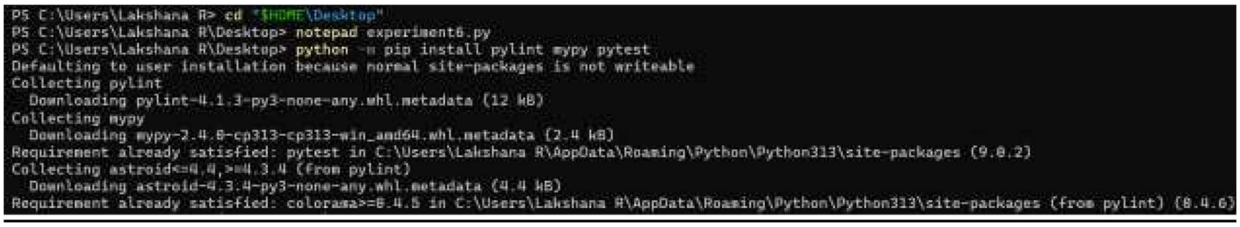
```
print("\n7. SUGGESTIONS FOR IMPROVEMENT")

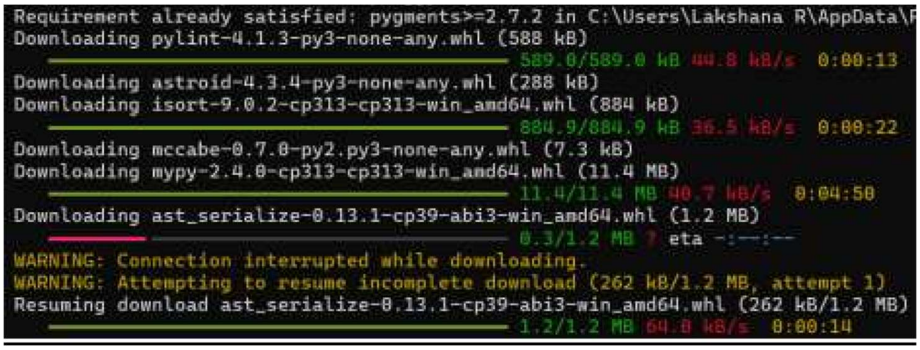
    for number, suggestion in enumerate(
        result["suggestions"], start=1
    ):
        print(f"{number}. {suggestion}")

    print("=" * 60)


# Run the complete framework
if __name__ == "__main__":
    evaluation = evaluate_code(
        generated_code, test_cases
    )
    print_report(generated_code, evaluation)
          print("URK23CS8003")
```

Output Screenshots:





```
[notice] A new release of pip is available: 26.2.1 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Lakshana R\Desktop> python -m pylint --version
pylint 4.1.3
astroid 4.3.4
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
PS C:\Users\Lakshana R\Desktop> python -m mypy --version
mypy 2.4.0 (compiled: yes)
PS C:\Users\Lakshana R\Desktop> python -m pytest --version
pytest 9.0.2
PS C:\Users\Lakshana R\Desktop> cd "$HOME\Desktop"
PS C:\Users\Lakshana R\Desktop> python experiment6.py
C:\Python313\python.exe: can't open file 'C:\\Users\\Lakshana R\\Desktop\\experiment6.py
PS C:\Users\Lakshana R\Desktop> cd "$HOME\Desktop"
PS C:\Users\Lakshana R\Desktop> python experiment6.py
C:\Python313\python.exe: can't open file 'C:\\Users\\Lakshana R\\Desktop\\experiment6.py
PS C:\Users\Lakshana R\Desktop> notepad "$HOME\Desktop\experiment6.py"
PS C:\Users\Lakshana R\Desktop> cd "$HOME\Desktop"
PS C:\Users\Lakshana R\Desktop> python experiment6.py
============================================================
AI-GENERATED CODE QUALITY EVALUATION REPORT
============================================================
```

```
1. SOURCE CODE
def add_numbers(a: int, b: int) -> int:
    """Return the sum of two numbers."""
    return a + b

2. SYNTAX CHECK
Syntax is valid.

3. PYLINT STATIC ANALYSIS
Score: 5.0 / 10
************* Module generated_code
C:\Users\LAKSHA~1\AppData\Local\Temp\tmpwon4lnra\generated_code.py:1:0: C0114: Missing module docstring (missing-module-docstring)

-----------------------------------
Your code has been rated at 5.00/10

4. MYPY TYPE CHECKING
Passed: True
Success: no issues found in 1 source file

5. DYNAMIC TESTING
PASS: add_numbers(2, 3) == 5
PASS: add_numbers(10, 20) == 30
PASS: add_numbers(-2, 2) == 0
PASS: add_numbers(0, 0) == 0

Tests passed: 4/4

6. OVERALL DEMONSTRATION SCORE
90.0/100

7. SUGGESTIONS FOR IMPROVEMENT
1. Review Pylint warnings to improve code style and quality.

Note: Pylint/Mypy findings depend on tool versions and settings.
The score is an illustrative classroom rubric, not a standard benchmark.
============================================================
PS C:\Users\Lakshana R\Desktop>
```

Link:

https://drive.google.com/file/d/1LgaSV0Myza6QLvlbKTq23Zn_brDBuHA-/view?usp=sharing

Result:

Thus, the AI-Generated Code Quality and Correctness Evaluation Framework was implemented using Python. The program checks syntax using Python AST, performs static analysis using Pylint, checks types using Mypy, executes test cases, calculates a quality score out of 100, and generates automatic suggestions for code improvement.